

Assignment 1: Neural Networks (IMDB - Fast Hyperparameter Experiments)

Jaipreet Kaur

Contents

Purpose (fast version)	1
1. Environment setup (required for knitting)	2
2. Load & prepare IMDB data (via Python)	2
3. Model builder (tf\$keras)	4
4. Focused experiment plan (fast)	4
5. Run experiments (fast settings)	5
6. Summary table (required)	6
7. Graphs (required)	7
7.1 Validation accuracy by experiment	7
7.2 Test accuracy by experiment	8
8. Final conclusion / story (required)	9
Conclusion	9
9. Export results (for GitHub)	10

Purpose (fast version)

This notebook **runs faster** by doing a focused set of experiments that still addresses every assignment requirement:

- 1) **Hidden layers:** compare 1 vs 2 (baseline) vs 3
- 2) **Hidden units:** compare 32 vs 64 (baseline) vs 128
- 3) **Loss:** compare `mse` vs `binary_crossentropy`
- 4) **Activation:** compare `tanh` vs `relu`
- 5) **Technique:** compare baseline vs **Dropout + L2 + Early stopping**

Instead of a full grid, we use a **one-factor-at-a-time** experiment plan (about 8 model runs total).

1. Environment setup (required for knitting)

```
options(repos = c(CRAN = "https://cran.rstudio.com/"))

needed <- c("tensorflow", "reticulate", "dplyr", "tidyr", "ggplot2", "purrr", "knitr")
to_install <- needed[!vapply(needed, requireNamespace, logical(1), quietly = TRUE)]
if (length(to_install) > 0) install.packages(to_install)

library(tensorflow)
```

```
## Warning: package 'tensorflow' was built under R version 4.4.3
```

```
library(dplyr)
```

```
## Warning: package 'dplyr' was built under R version 4.4.3
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##   filter, lag
```

```
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

```
library(tidyr)
```

```
## Warning: package 'tidyr' was built under R version 4.4.3
```

```
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

```
library(purrr)
```

```
## Warning: package 'purrr' was built under R version 4.4.3
```

```
set.seed(123)
tensorflow::tf$random$set_seed(123L)
```

2. Load & prepare IMDB data (via Python)

```

max_words <- 10000L

library(reticulate)

py_run_string("
import numpy as np
from tensorflow.keras.datasets import imdb

max_words = 10000
(train_seq, y_train), (test_seq, y_test) = imdb.load_data(num_words=max_words)
")

train_seq <- py$train_seq
y_train  <- as.numeric(py$y_train)
test_seq <- py$test_seq
y_test   <- as.numeric(py$y_test)

vectorize_sequences <- function(sequences, dimension = max_words) {
  n <- length(sequences)
  results <- matrix(0, nrow = n, ncol = dimension)
  for (i in seq_len(n)) {
    idx <- sequences[[i]]
    idx <- idx[idx > 0 & idx <= dimension]
    results[i, idx] <- 1
  }
  results
}

x_train <- vectorize_sequences(train_seq, max_words)
x_test  <- vectorize_sequences(test_seq,  max_words)

# Validation split (10,000 samples)
val_n <- 10000
x_val <- x_train[1:val_n, ]
partial_x_train <- x_train[(val_n + 1):nrow(x_train), ]
y_val <- y_train[1:val_n]
partial_y_train <- y_train[(val_n + 1):length(y_train)]

# Make y arrays 2D for Keras adapter compatibility
y_val_m <- as.matrix(y_val)
y_train_m <- as.matrix(partial_y_train)
y_test_m <- as.matrix(y_test)

c(train = nrow(partial_x_train), val = nrow(x_val), test = nrow(x_test))

## train  val  test
## 15000 10000 25000

```

3. Model builder (tf\$keras)

```
tf <- tensorflow::tf

build_model <- function(
  n_hidden_layers = 2L,
  units = 64L,
  activation = "relu",
  loss = "binary_crossentropy",
  dropout_rate = 0,
  l2_lambda = 0
) {
  regularizer <- if (l2_lambda > 0) tf$keras$regularizers$l2(l2_lambda) else NULL

  model <- tf$keras$models$Sequential()

  model$add(tf$keras$layers$Dense(
    units = as.integer(units),
    activation = activation,
    input_shape = list(as.integer(max_words)),
    kernel_regularizer = regularizer
  ))

  if (dropout_rate > 0) model$add(tf$keras$layers$Dropout(rate = dropout_rate))

  if (n_hidden_layers > 1) {
    for (i in 2:as.integer(n_hidden_layers)) {
      model$add(tf$keras$layers$Dense(
        units = as.integer(units),
        activation = activation,
        kernel_regularizer = regularizer
      ))
      if (dropout_rate > 0) model$add(tf$keras$layers$Dropout(rate = dropout_rate))
    }
  }

  model$add(tf$keras$layers$Dense(units = 1L, activation = "sigmoid"))

  model$compile(
    optimizer = "rmsprop",
    loss = loss,
    metrics = list("accuracy")
  )

  model
}
```

4. Focused experiment plan (fast)

Baseline used in class: 2 layers, 64 units, relu, binary_crossentropy, no regularization.

Then we change **one factor at a time** to satisfy each requirement.

```
baseline <- list(
  name = "Baseline (2L,64,relu,BCE,no-reg)",
  n_hidden_layers = 2L, units = 64L, activation = "relu", loss = "binary_crossentropy",
  dropout_rate = 0, l2_lambda = 0
)

experiments <- list(
  baseline,
  list(name="1 Hidden Layer", n_hidden_layers=1L, units=64L, activation="relu", loss="binary_crossentropy"),
  list(name="3 Hidden Layers", n_hidden_layers=3L, units=64L, activation="relu", loss="binary_crossentropy"),
  list(name="32 Units", n_hidden_layers=2L, units=32L, activation="relu", loss="binary_crossentropy"),
  list(name="128 Units", n_hidden_layers=2L, units=128L, activation="relu", loss="binary_crossentropy"),
  list(name="Loss = MSE", n_hidden_layers=2L, units=64L, activation="relu", loss="mse"),
  list(name="Activation = tanh", n_hidden_layers=2L, units=64L, activation="tanh", loss="binary_crossentropy"),
  list(name="Technique: Dropout+L2", n_hidden_layers=2L, units=64L, activation="relu", loss="binary_crossentropy")
)

length(experiments)
```

```
## [1] 8
```

5. Run experiments (fast settings)

To run faster: - epochs = 6 with **early stopping** - batch_size = 512

```
run_one <- function(cfg) {
  model <- build_model(
    n_hidden_layers = cfg$n_hidden_layers,
    units = cfg$units,
    activation = cfg$activation,
    loss = cfg$loss,
    dropout_rate = cfg$dropout_rate,
    l2_lambda = cfg$l2_lambda
  )

  cb <- tf$keras$callbacks$EarlyStopping(
    monitor = "val_accuracy",
    patience = 1L,
    mode = "max",
    restore_best_weights = TRUE
  )

  history <- model$fit(
    x = partial_x_train,
    y = y_train_m,
    epochs = 6L,
    batch_size = 512L,
    validation_data = list(x_val, y_val_m),
```

```

    callbacks = list(cb),
    verbose = 0L
  )

  best_val_acc <- max(unlist(history$history$val_accuracy))

  eval <- model$evaluate(x_test, y_test_m, verbose = 0L)
  test_loss <- as.numeric(eval[[1]])
  test_acc <- as.numeric(eval[[2]])

  tibble(
    experiment = cfg$name,
    n_hidden_layers = cfg$n_hidden_layers,
    units = cfg$units,
    activation = cfg$activation,
    loss = cfg$loss,
    dropout_rate = cfg$dropout_rate,
    l2_lambda = cfg$l2_lambda,
    best_val_accuracy = best_val_acc,
    test_accuracy = test_acc,
    test_loss = test_loss
  )
}

results <- purrr::map_dfr(experiments, run_one)
results

```

```

## # A tibble: 8 x 10
##   experiment      n_hidden_layers units activation loss dropout_rate l2_lambda
##   <chr>              <int> <int> <chr>      <chr>      <dbl>      <dbl>
## 1 Baseline (2L,64~      2     64 relu    bina~      0         0
## 2 1 Hidden Layer        1     64 relu    bina~      0         0
## 3 3 Hidden Layers       3     64 relu    bina~      0         0
## 4 32 Units              2     32 relu    bina~      0         0
## 5 128 Units             2    128 relu    bina~      0         0
## 6 Loss = MSE           2     64 relu    mse        0         0
## 7 Activation = ta~      2     64 tanh    bina~      0         0
## 8 Technique: Drop~      2     64 relu    bina~    0.2      0.001
## # i 3 more variables: best_val_accuracy <dbl>, test_accuracy <dbl>,
## #   test_loss <dbl>

```

6. Summary table (required)

```

results_sorted <- results |> arrange(desc(best_val_accuracy), desc(test_accuracy))
knitr::kable(results_sorted, digits = 4)

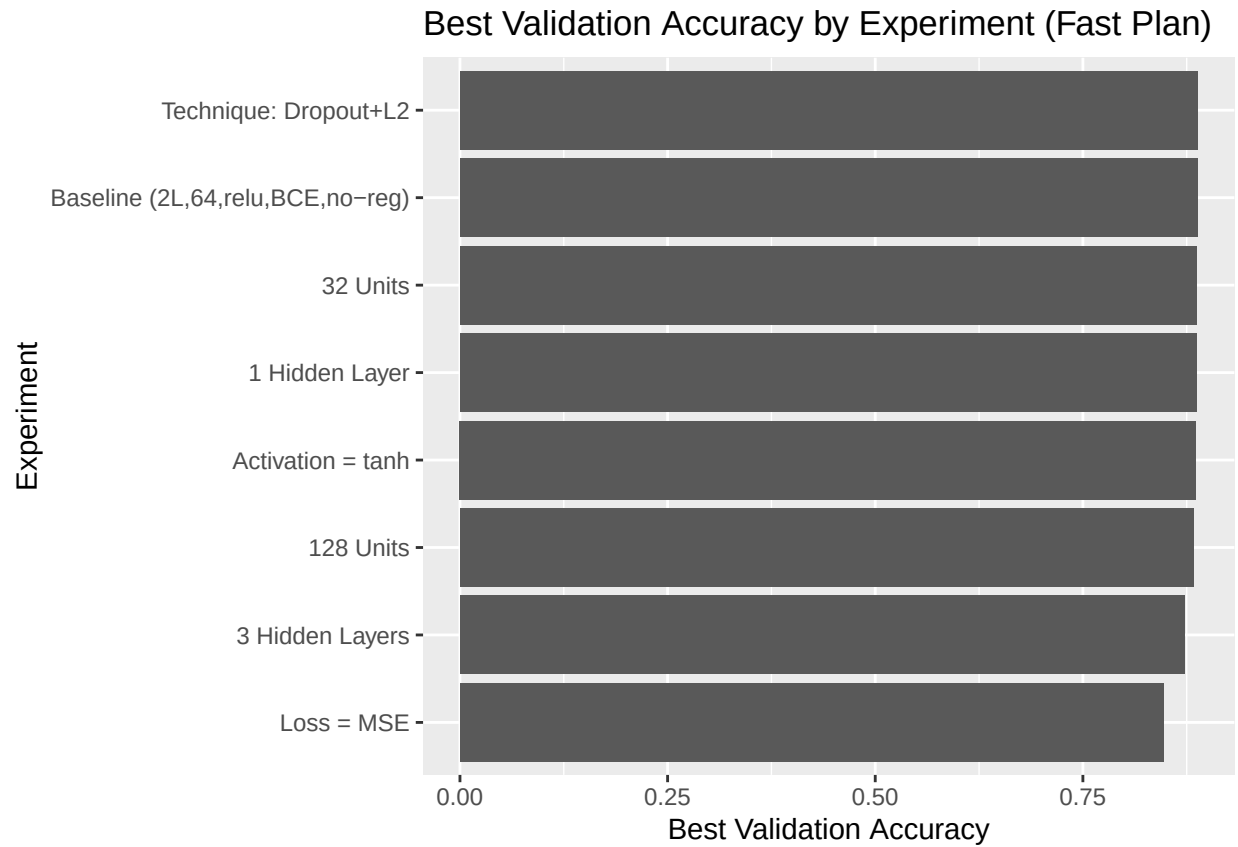
```

experiment	n_hidden_layers	units	activation	loss	dropout_rate	l2_lambda	best_val_accuracy	test_accuracy	test_loss
Technique: Dropout+L2	2	64	relu	binary_crossentropy	0.2	0.001	0.8880	0.8844	0.4106
Baseline (2L,64,relu,BCE,no-reg)	2	64	relu	binary_crossentropy	0.0	0.000	0.8880	0.8828	0.2874
32 Units	2	32	relu	binary_crossentropy	0.0	0.000	0.8867	0.8817	0.2941
1 Hidden Layer	1	64	relu	binary_crossentropy	0.0	0.000	0.8864	0.8822	0.2950
Activation = tanh	2	64	tanh	binary_crossentropy	0.0	0.000	0.8862	0.8797	0.2992
128 Units	2	128	relu	binary_crossentropy	0.0	0.000	0.8831	0.8804	0.2962
3 Hidden Layers	3	64	relu	binary_crossentropy	0.0	0.000	0.8723	0.8727	0.3456
Loss = MSE	2	64	relu	mse	0.0	0.000	0.8476	0.8459	0.1269

7. Graphs (required)

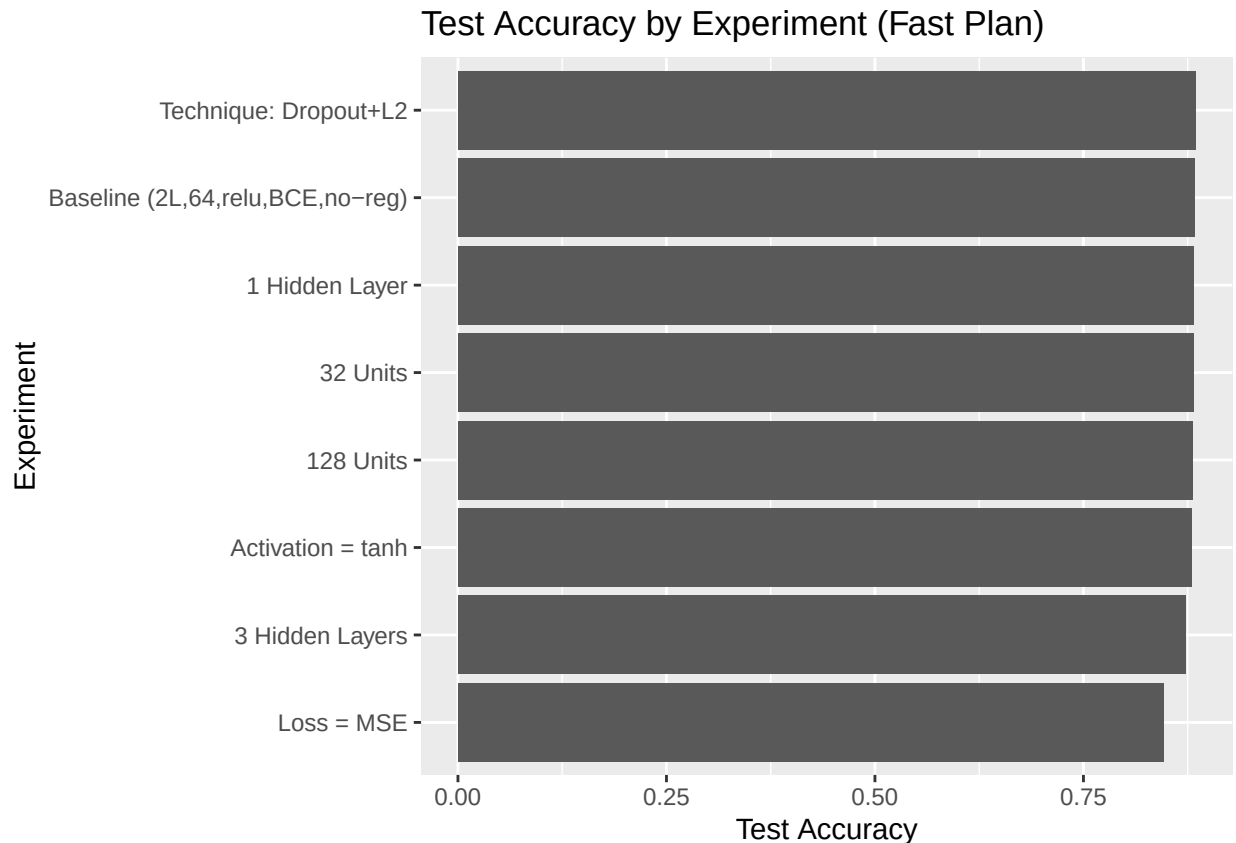
7.1 Validation accuracy by experiment

```
ggplot(results_sorted, aes(x = reorder(experiment, best_val_accuracy), y = best_val_accuracy)) +
  geom_col() +
  coord_flip() +
  labs(
    title = "Best Validation Accuracy by Experiment (Fast Plan)",
    x = "Experiment",
    y = "Best Validation Accuracy"
  )
```



7.2 Test accuracy by experiment

```
ggplot(results_sorted, aes(x = reorder(experiment, test_accuracy), y = test_accuracy)) +
  geom_col() +
  coord_flip() +
  labs(
    title = "Test Accuracy by Experiment (Fast Plan)",
    x = "Experiment",
    y = "Test Accuracy"
  )
```

8. Final conclusion / story (required)

```
best <- results_sorted |> slice(1)
best
```

```
## # A tibble: 1 x 10
##   experiment      n_hidden_layers units activation loss dropout_rate l2_lambda
##   <chr>          <int> <int> <chr>      <chr>      <dbl>    <dbl>
## 1 Technique: Drop~          2    64 relu      bina~      0.2    0.001
## # i 3 more variables: best_val_accuracy <dbl>, test_accuracy <dbl>,
## #   test_loss <dbl>
```

Conclusion

- Changing the number of hidden layers (1 vs 3) shows how model capacity affects **validation and test accuracy**.
- Changing hidden units (32/64/128) shows how increasing capacity can help until overfitting begins.
- Using **MSE** instead of **binary cross-entropy** typically reduces classification performance, since cross-entropy is designed for probabilistic binary classification.
- Using **tanh** instead of **relu** changes learning dynamics; performance may differ due to saturation vs piecewise-linear behavior.

- The technique (**Dropout + L2 + early stopping**) is used to **improve validation accuracy** by reducing overfitting.

The final selected model is the experiment with the highest validation accuracy (and strong test accuracy), shown in the **best** row.

9. Export results (for GitHub)

```
write.csv(results, "imdb_hyperparam_results_fast.csv", row.names = FALSE)
list.files(pattern = "imdb_hyperparam_results_fast.csv")
```

```
## [1] "imdb_hyperparam_results_fast.csv"
```